

INGENIERÍA DE SOFTWARE
DESARROLLO DE APLICACIONES OPEN SOURCE (1ASI0729)
EXAMEN PARCIAL
202620

NRC: 7760
Profesores: Bautista Ubillús, Efraín Ricardo
Flores Moroco, Juan Antonio
Mori Paiva, Hugo Allan
Robles Fernández, Iván
Velásquez Núñez, Ángel Augusto
Duración: 170 minutos
Indicaciones:

1. El examen consta de 1 pregunta, y tendrá 170 minutos para resolverlas.
2. La pregunta es de tipo Proyecto de Software y la entrega de su respuesta es a través de envío de archivo empaquetado **.zip** (único formato válido) con nombre ea<nrc>u<código-estudiante>.zip (por ejemplo, ea16692u201621873.zip), conteniendo el proyecto de software, en la Actividad para el Examen parcial.
3. Para el desarrollo del examen se requiere archivos de configuración que se encuentran como archivo adjunto (**upc-pre-202620-1asi0729-examen-parcial-7760_v1-files.zip**).
4. Debe crear su proyecto de solución y evidenciar la autoría.
5. Antes de la generación del archivo **.zip** para el envío, elimine la carpeta *node_modules*.
6. Control de Versiones y Autoría:
   - El desarrollo debe realizarse de forma incremental utilizando Git, aplicando estrictamente la especificación de Conventional Commits en inglés [ej. feat(proposals): add summary grid, fix(state-management): handle empty state].
   - Antes de iniciar la evaluación debe retiren la configuración global de Git:
     ```
     git config --global –unset user.name
     git config --global –unset user.email
     ```
   - Debe agregar al profesor de la NRC del curso (Usuario de GitHub: **ajaflorez-teacher**) como Administrador de su repositorio privado desde el inicio de la evaluación.
   - La URL del repositorio privado en GitHub debe quedar documentada explícitamente en la sección *Authorship* del archivo README.md del proyecto, luego del título y la sección *Overview*.
   - El historial cronológico de commits y la actividad en el repositorio son parte obligatoria de la evidencia de autoría independiente durante el horario de la evaluación. Un único commit masivo previo a finalizar la evaluación, o un número muy reducido de commits sin coherencia con la evolución de construcción de la solución, o commits fuera del horario de evaluación (antes o después), invalidará la entrega.
7. Puede utilizar como referencia los materiales publicados en el aula virtual, los sitios web de documentación de frameworks, lenguajes de programación utilizados, así como ejemplos de clase (solo como referencia, no como fuente de duplicado o copia de archivos).
8. **Restricción de IA:** No se permite AI Agentic Development de ningún tipo. Solo se permite el uso del asistente del IDE de JetBrains para completar instrucciones durante la edición.

Enunciado:

Pregunta 1 (20 p.).

Caso Tesla Supercharger Fleet & Battery Storage Platform

Tesla (https://www.tesla.com), pionera mundial en vehículos eléctricos y almacenamiento de energía limpia, busca mantener un control estricto y centralizado sobre la capacidad operativa de sus estaciones Supercharger, la demanda de carga en tiempo real y la disponibilidad de conectores de alta potencia. Ante la expansión masiva de su red de carga y la integración de nuevos estándares de conectividad, Tesla se ha propuesto implementar una plataforma web moderna para la gestión logística de carga de flotas y monitoreo de subestaciones eléctricas.

La evaluación tiene un empaquetado adjunto conteniendo un archivo, el cual puede ser usado por json-server (https://github.com/typicode/json-server/tree/v0) stable version 0.17.4 para simular un backend con las características indicadas.

Al ejecutar json-server en el terminal, utilizando el archivo de configuración mencionado (debe ubicar el archivo en la carpeta *server* de su proyecto), con los comandos:

```
cd server
json-server --watch db.json
```

Esto inicia el Fake API.

La información general de las **ChargingStations** (*id, stationName, vehicleType, maxGridKwCapacity, stallsAvailable, isActive, locationCoordinates*) se encuentra en el endpoint:

`http://localhost:3000/charging-stations`

La información de las **VehicleCharges** (*id, stationId, vehicleType, makeId, makeName, kwDemand, chargingPortType, isFastCharge, registeredAt*) se encuentra en el endpoint:

`http://localhost:3000/vehicle-charges`

Notas sobre el external API Provider:

La solución incluye la integración con la API pública de la *NHTSA Vehicle API* (https://vpic.nhtsa.dot.gov/api/). Los valores para *makeId* y *makeName* se obtienen de la respuesta enviada por el servicio a través del endpoint:
`https://vpic.nhtsa.dot.gov/api/vehicles/GetMakesForVehicleType/{`**`vehicleType`**`}?format=json`

Los valores para *vehicleType* pueden ser "`car`", "`truck`", "`multipurpose`", "`motorcycle`". Estos tipos coinciden con los parámetros de filtrado del endpoint público de la NHTSA.

Los resultados que ofrece ese endpoint contiene un JSON Arrays con objetos con la siguiente estructura (se muestra la estructura con datos de ejemplo):

```
{
  "Count": 195,
  "Message": "Response returned successfully",
  "SearchCriteria": "Vehicle Type: car",
  "Results": [
    {
      "MakeId": 440,
      "MakeName": "ASTON MARTIN",
      "VehicleTypeId": 2,
      "VehicleTypeName": "Passenger Car"
    },
```

```
    {
      "MakeId": 441,
      "MakeName": "TESLA",
      "VehicleTypeId": 2,
      "VehicleTypeName": "Passenger Car"
    },
    …
  ]
}
```

Para el desarrollo web de lado web frontend, se ha seleccionado TypeScript como lenguaje de programación y Angular como Frontend Framework.

Se le encarga el desarrollo de una aplicación web que implemente las siguientes características:

- Un Toolbar, donde se muestra a la izquierda el logo institucional de *Tesla* (para ello utilice el servicio de Logo.dev Logo API con el dominio tesla.com) y a continuación el texto “Tesla Supercharger & Fleet Platform”.
- El toolbar debe ofrecer a la derecha de las opciones del toolbar, select buttons con “EN” y “ES” para el switching de idioma.
- Se incorpora un Sidebar (Sidenav) lateral basado en Angular Material que contiene las opciones de navegación: "Home" y "New Vehicle Charge".

- La vista **Home** muestra como título el texto “Home” y como contenido el texto “*Intelligent Fleet Power Distribution and Supercharger Management*.”
- Dentro de la vista **Home** se muestra también una sección: “Charging Stations”.
- La sección con el título “*Charging Stations*” presenta un grid list de dos componentes Station Summary por columna. Cada tarjeta (**card**) debe presentar en su header el nombre de la estación (*stationName*). En la sección de contenido (*content*) se debe listar para cada carga activa de vehículo registrada su marca (*makeName*), la demanda de potencia (kwDemand kW) y el tipo de conector (*chargingPortType*). Si el registro tiene *isFastCharge* en true, se debe desplegar una etiqueta visual o chip distintivo. En el footer de la tarjeta deben aparecer los indicadores: **Total Grid Load, Available Grid Margin y Active Stalls Ratio**.
- El indicador **Total Grid Load** presenta la suma de *kwDemand* para todos los vehículos cargando en la estación.
- El indicador **Available Grid Margin** muestra la resta de *maxGridKwCapacity* menos el **Total Grid Load** de la estación.
- El indicador **Active Stalls Ratio** muestra la cantidad de cargas registradas respecto a los conectores físicos disponibles (*stallsAvailable*), en formato "X / Y Stalls".
- Si no se han registrado cargas (vehicle-charges) para una estación, el contenido debe mostrar "Empty".

- La vista **New Vehicle Charge** muestra en la parte superior el título “New Vehicle Charge” y debajo un subtítulo "*Register Electric Vehicle Power Demand to Station*".
- La vista **New Vehicle Charge** presenta un formulario reactivo que permite al usuario: Seleccionar el tipo de vehículo (*vehicleType*), seleccionar la marca del vehículo (*makeId y makeName*) a partir de la colección que ofrece la NHTSA API para ese tipo, seleccionar el tipo de conector (*chargingPortType*, con opciones como "NACS", "CCS"), especificar la demanda de potencia en kW (*kwDemand*) e Indicar si es carga rápida (*isFastCharge*) mediante un toggle switch.
- El atributo *registeredAt* no debe solicitarse en el formulario, sino poblarse automáticamente (con la fecha y hora actual) al crear el registro. El *id* se genera automáticamente. Ofrezca al usuario las acciones de Create y Cancel. Ambas llevan al usuario a la vista principal (/home).
- La estación (*stationId*) a la que debe asociarse el registro de carga se determina automáticamente según el tipo de vehículo (hay una única estación por cada *vehicleType*).
- Solo se puede registrar un VehicleCharge cuando se cumplen simultáneamente las siguientes condiciones de negocio: La estación asociada está activa (*isActive* = true), la cantidad de vehículos cargando actualmente es estrictamente menor a la cantidad de conectores disponibles (*stallsAvailable*) y la demanda de potencia (*kwDemand*) es menor o igual que el **Available Grid Margin** calculado para la estación en ese momento.

- Considere una vista de tipo *page-not-found* para el caso de rutas de navegación no soportadas. Dicha vista debe mostrar un mensaje incluyendo la ruta especificada que no se encontró y debe ofrecer un botón para retornar a *Home*.
- La vista Home es accesible desde /home.
- La vista **New Vehicle Charge** es accesible desde /charges/new.
- La ruta raíz (/) debe redirigir al usuario a /home.
- La interfaz de usuario debe estar disponible en inglés y español, siendo **inglés** el idioma por defecto.
- La interfaz de usuario debe tener aproximación con el design system del website de *Tesla*. Para ello puede utilizar uno de los pre-built palettes de Angular Material que tenga mayor similitud y adaptar algunas características (ver sección de referencias).

El equipo de IT de su cliente tomará en cuenta no sólo el cumplimiento de las características funcionales, sino el diseño de interfaz de usuario, así como la estructura del proyecto, aplicación de convenciones de nomenclatura de objetos de programación en inglés, convenciones de nomenclatura de Angular, organización y eficiencia del código. Igualmente se tomará en cuenta la aplicación de patrones de diseño. Se toma en cuenta accessibility (uso de ARIA attributes) y usability.

Restricciones técnicas:

Nombre su proyecto como *ea**nrcu**code* donde ***code*** es su código de estudiante en minúsculas (por ejemplo, *ea7760u201621873*). Cree su proyecto usando **Angular Framework** v**22.2.2** o superior. **Todos** los componentes de su proyecto deben ser de tipo *stand-alone components*.
El equipo requiere que la interfaz de usuario esté basada estrictamente en **Angular Material**, mientras que para la comunicación con el backend debe apoyarse en **HttpClient** (incluido en @angular/common/http). La aplicación debe soportar in-app navigation y utilizar **@angular/router** para el manejo de routing en la aplicación, aplicando semantic routes e incluyendo child routes por bounded context. De requerirse, utilice **DatePipe** en los templates para los formatos de date (ver sección de referencias). Debe incluir ARIA atributes en las vistas. La interfaz de usuario debe mostrar los textos en **inglés** por defecto. Debe utilizar **@ngx-translate/core @ngx-translate/http-loader** y para la implementación de i18n. Utilice **Angular Signals** para la implementación de State Management.
El proyecto de aplicación debe elaborarse en JetBrains IntelliJ Idea o WebStorm.
La organización del proyecto debe ser domain-driven aplicando layered y component-based architecture, object-oriented programming y design patterns, considerando los sub-dominios ***shared*** (para elementos base o de uso común en otros sub-dominios), ***stations*** (para componentes o elementos relacionados con las estaciones de carga y consumos) y **vehicles** (para componentes o elementos relacionados con la información que proporciona el external API Provider de la NHTSA). Aplique buenas prácticas para nomenclatura lógica y física de clases y componentes. Distribuya adecuadamente los elementos dentro de cada carpeta de sub-dominio, considerando como primer nivel de sub-carpetas para cada bounded context: *domain*, *application*, *infrastructure*, *presentation*, considerando según el caso sub-niveles como *views*, *components*, *model*. En *views* solo se ubican componentes que tienen relación directa con rutas de navegación. En *components* se ubican componentes que son incluidos en otros y no están asociados de forma directa a rutas de navegación. Para los identificadores de textos de traducción de idiomas, utilice kebab-case. Para referenciar a los componentes dentro de templates utilice kebab-case. Utilice kebab-case para identificar los translation strings. Aplique patrones de diseño, incluyendo *Request/Response*, *Resource*, *Assembler, Api, Endpoint y Store*. En caso de que la información de los JSON objects que proporciona el API provider, incluidos en los response no cumplan con las convenciones de nomenclatura de objetos de TypeScript, ello no debe afectar la nomenclatura de los domain entities. Es responsabilidad de los Assembler objects la asignación de información a los atributos que correspondan entre resource objects y domain objects. Utilice environment variables para evitar hard coding de URLs o PATHs. Comente los archivos de código fuente en TypeScript elaborados por usted utilizando las convenciones de **TSDoc** (ver referencias), con block comments incluyendo un texto de *summary* con el propósito y *author* con su nombre y apellido, así como documentación para clases y sus principales elementos, todo ello en inglés.
Incluya en el archivo README.md, la información en inglés de la aplicación, descripción, información de features, stack y configuración. Incluya en README.md una sección *Authorship*, conteniendo su información como autor y el URL del repositorio privado en GitHub.

Antes de la generación del archivo **.zip** (único formato válido) para el envío, elimine la carpeta *node_modules*. El nombre del archivo .zip debe seguir la estructura *ea**nrcu**code.zip* (por ejemplo, *ea7760u201821873.zip*).

Nota: Cree su proyecto. No se permite refactoring de proyectos previos. Solo se considerará aptas para revisión y calificación soluciones con archivos creados integramente durante el horario de la evaluación. Solo se permite el uso del asistente del IDE de JetBrains para completar instrucciones durante la edición. No se permite AI Agentic Development de ningún tipo. Debe cumplir las indicaciones sobre Control de Versiones y Autoría.

Referencias:

Materiales del curso
https://www.geeksforgeeks.org/typescript/how-to-calculate-the-time-between-2-dates-in-typescript/
https://ngx-translate.org/
https://material.angular.dev/guide/theming
https://angular.dev/guide/routing/common-router-tasks
https://angular.dev/guide/routing/router-reference
https://angular.dev/guide/routing/router-tutorial#adding-a-404-page
https://angular.dev/guide/http
https://angular.dev/guide/templates/pipes
https://angular.dev/api/common/DatePipe
https://material.angular.dev/components/grid-list/overview
https://material.angular.dev/components/card/overview
https://material.angular.dev/components/card/overview#accessibility
https://material.angular.dev/components/toolbar/overview
https://material.angular.dev/components/toolbar/overview#accessibility
https://github.com/typicode/json-server/tree/v0
https://www.w3.org/TR/wai-aria/#usage
https://github.com/ngx-translate/core
https://tsdoc.org/
https://jsdoc.app/

Rúbrica de calificación[1]

Criterio de Calificación	Sobresaliente (S)	Esperado (E)	Necesita Mejorar (M)	Insuficiente (I)	Calificación
C01. Building y ejecución	Al abrir el proyecto y ordenar la ejecución, ésta se inicia sin problemas. La aplicación es accesible en la ruta indicada.	La aplicación no llega a iniciar y ejecutarse, sin embargo el proceso de building llega a concluir.	Al cargar el proyecto el proceso de building presenta errores y no llega a concluir.	No elabora solución	
	2.0 puntos	**1.25 punto**	**0.5 puntos**	**0 puntos**	
C02. User Interface & Home View	Se evidencia que la interfaz de usuario aplica Responsive Web Design. La interfaz cumple con las características solicitadas para la estructura, elementos de la interfaz de usuario, rutas de navegación, idioma e internalización. Se evidencia que la aplicación recupera y presenta en Home la información solicitada según especificaciones y presenta una vista de tipo page-not-found ante rutas de navegación no soportadas. Adicionalmente presenta soporte completo para i18n.	La interfaz de usuario no evidencia con claridad que aplica Responsive Web Design, o no cumple con todas las características solicitadas, sin embargo, presenta el Toolbar con las opciones solicitadas, recupera y presenta en Home la información solicitada.	La aplicación no implementa una interfaz de usuario con las características solicitadas que aplique Responsive Web Design aunque presenta parte de la información indicada.	La aplicación no presenta la información indicada.	
	5.0 puntos	**3.5 puntos**	**1.5 puntos**	**0 puntos**	
C03. Features	Se evidencia que la aplicación proporciona las opciones solicitadas, con el comportamiento esperado, con validaciones, control de errores de interacción o navegación, con comunicación adecuada hacia el usuario. La vista cumple con los requisitos de interfaz de usuario, ruta de navegación, con todas las características y restricciones indicadas. Adicionalmente presenta soporte completo para i18n.	Se evidencia que la aplicación proporciona las opciones solicitadas, llevando a la ruta de navegación solicitada, con el comportamiento esperado, pero aplica de forma parcial validaciones, o controla de forma parcial los errores de interacción o navegación, o no comunica de forma adecuada al usuario, o cumple de forma parcial con los requisitos de interfaz de usuario, ruta de navegación, o cumple con parte de las características y restricciones indicadas.	La aplicación presenta las vistas en la ruta de navegación, pero no presenta la información solicitada o incumple la mayoría de características y comportamiento solicitados.	No se implementa la opción.	
	7.0 puntos	**4.5 puntos**	**2 puntos**	**0 puntos**	
C04. Code Organization	El desarrollador organiza el código y los elementos de frontend de la solución, aplicando buenas prácticas de TypeScript y Angular, agrupando los elementos de la solución según convenciones, manteniendo organización de paquetes y carpetas recomendadas por el fabricante y buenas prácticas de la industria de software, bajo un enfoque domain-driven, cumpliendo con las restricciones técnicas sobre organización.	El desarrollador aplica parcialmente en frontend convenciones, recomendaciones y buenas prácticas de TypeScript o Angular, o no cumple con todas las restricciones técnicas sobre organización.	El desarrollador aplica en frontend solo algunas de las convenciones, recomendaciones y buenas prácticas de TypeScript o Angular, o no cumple con las restricciones técnicas sobre organización.	No se evidencia un criterio de organización para los elementos de la solución, limitándose a la estructura por defecto.	
	2.0 punto	**1.25 puntos**	**0.5 puntos**	**0 puntos**	
C05. Code Quality	Utiliza para el frontend el lenguaje de programación TypeScript, el framework Angular y cumple con las restricciones técnicas indicadas. La codificación tiene un estilo claro, indentando los bloques de código según los estándares de programación correspondientes al lenguaje, aplicando una lógica consistente en los métodos, condicionales sin escenarios no contemplados, uso adecuado de reutilización de código para evitar redundancia. Aplica patrones de arquitectura y patrones de diseño. Distribuye el código en los niveles correspondientes, asignando lógica de persistencia, lógica de negocio, lógica de control, y transferencia a las interfaces y clases que corresponden.	Utiliza para el frontend el lenguaje de programación TypeScript y el framework Angular. La codificación es funcional, pero cumple de forma parcial con las restricciones técnicas, o sólo aplica parcialmente los estándares de indentación de bloques de código, ó existen ineficiencias en la codificación: redundancia ó inconsistencias en la lógica de programación. Aplica parcialmente patrones de arquitectura y patrones de diseño, o existe en algunas partes una distribución de la lógica en los niveles incorrectos.	Cumple con solo algunas restricciones técnicas, o no se evidencia aplicación de estándares ó criterios de eficiencia en la codificación, con ausencia de comentarios, ó no aplica patrones de arquitectura ni patrones de diseño, aunque la codificación es funcional.	No utiliza el lenguaje de programación TypeScript o no utiliza Angular, ó no cumple con la mayoría de restricciones técnicas o el código no es funcional.	
	3.0 puntos	**2.0 punto**	**1.0 puntos**	**0 puntos**	
C06. Naming Standards	El desarrollador aplica en todos los nombres de objetos de programación como paquetes, componentes, interfaces, clases, objetos, variables, constantes y métodos la nomenclatura en inglés y la nomenclatura estándar para identificadores de clases, objetos, miembros de programación, así como los recursos.	El desarrollador aplica en la mayoría de casos la nomenclatura en inglés y la nomenclatura estándar para identificadores de clases, objetos, miembros de programación, así como los recursos.	El desarrollador aplica sólo en algunos casos la nomenclatura en inglés y la nomenclatura estándar para identificadores de clases, objetos, miembros de programación, así como los recursos.	El desarrollador no aplica nomenclatura en inglés para los objetos de programación ó recursos.	
	1.0 puntos	**0.5 punto**	**0.25 puntos**	**0 puntos**	
Total	**20 puntos**	**13.0 puntos**	**5.75 puntos**	**0 puntos**	

Lima, 14 de octubre del 2026

[1] Solo aplica para soluciones válidas